

Stack Exploitation: Overflow to Shellcode

The old story (x86, no mitigations)

```
char buf[16]; gets(buf);    # no bounds
-> write 16+8 (saved rbp) + 8 (ret) = 32 -> then smash RET with shellcode addr
0*... shellcode ... addr_of_buf    (ret slides into the NOP sled)
```

That was 2005-era. The world got mitigations.

The mitigation quartet

Mitigation	Blocks	Bypass
ASLR	predictable addresses	info leak / brute force
NX (DEP, W^X)	shellcode on stack	ret2libc
CANARY	overwrite detection	leak the canary / other bugs
PIE (RELRO)	unknown base	leaks or one-gadget

Modern basics (amd64)

Stack protector (canary) on by default in GCC: `-fstack-protector`

NX on by default: executing stack = segfault

ASLR ON: libc/stack addresses random per run

```
# compile targets (lab):
gcc -m32 -no-pie -fno-stack-protector -z execstack -o vuln vuln.c    (classic NOP-sled target)
gcc -no-pie -fno-stack-protector -o vuln2 vuln.c                    (NX on -> ret2libc)
```

The workflows

Classic (execstack): find offset (cyclic), fill to ret, put address of buffer/NOP sled → jump → shell.

Ret2libc (NX only): overflow ret area with `system("bin/sh")` chain – no shellcode needed:

```
ret -> system@libc ; ret ; "sh" crude
```

ASLR blocks exact libc addresses unless you leak them (printf of a pointer is the classic leak).

Full modern: leak canary + leak libc base → chain ROP → win. Now we're pros.

Explains the craft

Use pwntools (Python) for byte-precise payloads:

```
from pwn import *
p = process('./vuln')
p.sendline(cyclic(200)); p.wait()
core = p.corefile                # find offset
```

Ethics ready

This is exploit development ON YOUR OWN LAB BOXES, with pwn.college/CTF binaries – the discipline that becomes defensive knowledge (you must know the attack to tune the DEFENSE – see M36/M39 chain).

The overflow is not dead; it's armored. Ret2libc/ROP are the ways the modern attacker undoes NX+ASLR. Practicing exploit dev on CTFs is how you earn your blue-belt in defense.

[← Debugging with GDB: Breakpoi...](#)

[Ret2libc, ROP & Chains →](#)